lec512

Dylan Tang

May 2026

1 Principal Component Analysis

- PCA finds a new, lower-dimensional basis that attempts to capture as much variance in the data as possible
- Recall we can find PCA of rank k with SVD.

$$X_k = U_k \Sigma V_k^T$$

- PCA is useful in that the degrees of freedom of plausible values is often much larger than the degrees of freedom that actually make sense in the real world.
- **Information Bottleneck Theory** - All significant learning requires setting an information bottleneck
  - Information bottleneck - real-world data often lies in some lower dimensional space when compared to the entire subset of possible data values

2 Neural Networks

2.0.1 Forward Propagation

- Consider input observations $X_1, \ldots, X_n$, where each $X_i \in \mathbb{R}^p$.
- At each hidden layer, compute a weighted linear combination of the inputs, then apply a nonlinear activation function:

$$a_i^{(j)} = \sigma\left(\left[w^{(j)}\right]^T X_i + b^{(j)}\right),$$

where $w^{(j)}$ is the weight vector for hidden unit j, $b^{(j)}$ is the bias term, and $\sigma(\cdot)$ is the activation function.

- More generally, for layer ℓ, the forward propagation step is

$$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)},$$
$$A^{(\ell)} = \sigma\left(Z^{(\ell)}\right),$$

  where $A^{(0)} = X$ is the input layer.

- The final layer produces the network output:

$$\hat{Y} = A^{(L)}.$$

- At the final layer, the network produces a prediction $\hat{Y}_i$ for each input X_i:

$$\hat{Y}_i = f(X_i; \theta),$$

  where θ represents all weights and biases in the network.

2.0.2 Gradient Descent

Gradient descent is given by:

$$W^{(t+1)} = W^{(t)} - \eta \nabla_W \mathcal{L}\left(W^{(t)}\right)$$

2.0.3 Backward Propogation

- For a hidden layer ℓ, recall that

$$Z^{(\ell)} = W^{(\ell)} A^{(\ell-1)} + b^{(\ell)}, \qquad A^{(\ell)} = \sigma\left(Z^{(\ell)}\right).$$

Using the chain rule, the gradient with respect to $W^{(\ell)}$ is

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial A^{(\ell)}} \frac{\partial A^{(\ell)}}{\partial Z^{(\ell)}} \frac{\partial Z^{(\ell)}}{\partial W^{(\ell)}}.$$

Since

$$\frac{\partial A^{(\ell)}}{\partial Z^{(\ell)}} = \sigma'\left(Z^{(\ell)}\right), \qquad \frac{\partial Z^{(\ell)}}{\partial W^{(\ell)}} = \left(A^{(\ell-1)}\right)^T,$$

we get

$$\boxed{\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \left[\frac{\partial \mathcal{L}}{\partial A^{(\ell)}} \odot \sigma'\left(Z^{(\ell)}\right)\right]\left(A^{(\ell-1)}\right)^T}$$

Recall that the recurrence relation is

$$A^{(\ell)} = \sigma\left(W^{(\ell)} A^{(\ell-1)} + b^{(\ell)}\right)$$

so we could find $\frac{\partial \mathcal{L}}{\partial A^{(\ell)}}$ by repeatedly unwrapping the recurrence relation.

Remarks

- Gradient descent requires optimizing for each weight connecting nodes from ℓ to $\ell + 1$ - expensive
- It is important to standardize $X_1 \dots X_N$ to prevent exploding/vanishing gradients

2.0.4 Code Snippets

Standard Regression using Neural Nets

```
model = tf.keras.models.Sequential() // standard layer-by-layer NN
model.add(tf.keras.layers.InputLayer(input_shape=(8,))) // X1...X8
model.add(tf.keras.layers.Dense(units=1)) // one hidden layer with no activation

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.MeanSquaredError()
)

history = model.fit(
    x=x,
    y=y,
    batch_size=1000,
    epochs=epochs,
    shuffle=True
)
```

2 Dense Layers, ReLu activation

```
model = tf.keras.models.Sequential()

model.add(tf.keras.layers.InputLayer(input_shape=(8,)))

model.add(tf.keras.layers.Dense(units=16, activation='relu'))
model.add(tf.keras.layers.Dense(units=8, activation='relu'))

model.add(tf.keras.layers.Dense(units=1))

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.MeanSquaredError()
)

history = model.fit(
    x=x,
    y=y,
    batch_size=1000,
    epochs=epochs,
    shuffle=True
)
```

- **Adam** optimizatin is gradient descent with momentum and adaptive learning rates.

Convolutional Neural Network

- Instead of having only activation functions as bottlenecks, has more complex hidden layers with pooling, stride, padding, and filter.

```
import tensorflow as tf

model = tf.keras.models.Sequential()

# Input image: 28 x 28 grayscale image
model.add(tf.keras.layers.InputLayer(input_shape=(28, 28, 1)))

# First convolution block
model.add(tf.keras.layers.Conv2D(
    filters=32,
    kernel_size=(3, 3),
    strides=(1, 1),
    padding='same',
    activation='relu'
))

# Pooling layer
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2),
    strides=(2, 2)
))

# Second convolution block
model.add(tf.keras.layers.Conv2D(
    filters=64,
    kernel_size=(3, 3),
    strides=(1, 1),
    padding='same',
    activation='relu'
))

# Pooling layer
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(2, 2),
    strides=(2, 2)
))

# Flatten feature maps into vector
model.add(tf.keras.layers.Flatten())

# Fully connected layer
model.add(tf.keras.layers.Dense(
```

```
    units=128,
    activation='relu'
))

# Output layer for 10-class classification with softmax loss
model.add(tf.keras.layers.Dense(
    units=10,
    activation='softmax'
))

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(),
    metrics=['accuracy']
)

history = model.fit(
    x=x_train,
    y=y_train,
    batch_size=64,
    epochs=epochs,
    shuffle=True,
    validation_data=(x_test, y_test)
)
```